

Problem 1

In problem 2 (b) I just stated that $d_i = \frac{d_{i-1} + 1}{2} = \frac{d + 2^n - 1}{2^n}$, which I could have improved on by showing my work on how I got that relation. I can fix it by using induction on this and showing why you would get this relation.

Problem 2

- (a) While we have to do $O(b)$ multiplications the problem is b is the value of the number, and we would actually have to do $O(2^k)$ operations where k is the number of bits used in b thus this algorithm is not efficient.
- (b) Let $p, q, m \in \mathbb{Z}_{>0}$ s.t. $m \geq 2$. By definition of modulo we can write p, q in the form of:

$$p = km + (p \bmod m), \quad q = lm + (q \bmod m)$$

s.t. $k, l, (p \bmod m), (q \bmod m) \in \mathbb{Z}$. Thus, consider:

$$\begin{aligned} pq &= (km + (p \bmod m))(lm + (q \bmod m)) \\ &= klm^2 + km(q \bmod m) + lm(p \bmod m) + (p \bmod m)(q \bmod m) \end{aligned}$$

Now we can look at the equivalence under mod m . Thus:

$$pq \equiv (p \bmod m)(q \bmod m) \bmod m$$

The statement is:

$$\text{if } b \text{ is even, then } (a^{b/2} \bmod m)^2 \bmod m = a^b \bmod m$$

This proof comes directly from the one above since $a^{b/2} \cdot a^{b/2} = a^b$. Also, we can divide b by two since it is even.

Algorithm 1 Fast Modular Exponentiation(c) **Require:** Positive integers a , b , and $m \geq 2$.**Ensure:** $a^b \bmod m$.

```

1: function MODULAR( $a, b, m$ )
2:   if  $b = 0$  then
3:     return 1 ▷ Base case:  $a^0 = 1$ 
4:   if  $b = 1$  then
5:     return  $a \bmod m$  ▷ Base case:  $a^1 = a \bmod m$ 
6:   if  $b$  is even then
7:      $\text{temp} \leftarrow \text{MODULAR}(a, b/2, m)$  ▷ Recursively compute  $a^{b/2} \bmod m$ 
8:     return  $\text{temp}^2 \bmod m$  ▷ Now it becomes  $a^b \bmod m$ 
9:   if  $b$  is odd then
10:     $\text{temp} \leftarrow \text{MODULAR}(a, (b-1)/2, m)$  ▷ Recursive call for  $a^{(b-1)/2} \bmod m$ 
11:     $\text{temp} \leftarrow (\text{temp}^2) \bmod m$  ▷ Now it is  $a^{(b-1)} \bmod m$ 
12:    return  $(\text{temp} \cdot (a \bmod m)) \bmod m$  ▷ Now it becomes  $a^b \bmod m$ 

```

- (Proof of Correctness)

Base Cases ($b = 0, 1$):If $b = 0, 1$ then $a^b \bmod m$ is either $a^0 = 1$ for $b = 0$ and $a^1 \bmod m$ for $b = 1$

Inductive Step:

Assume that the algorithm works for all $1 < b \leq n$ for some $n \in \mathbb{N}$. Consider $n + 1$.If $n + 1$ is even then using the proof in (b) we know that $(a^{(n+1)/2} \bmod m)^2 \bmod m = a^{n+1} \bmod m$, so by the induction hypothesis since $(n + 1)/2 \leq n$ we recursively call on $a^{(n+1)/2}$ and then use the formula in (b) to get $a^{n+1} \bmod m$.If $n + 1$ is odd then this implies that n is even thus by the induction hypothesis since $n/2 \leq n$ we recursively call on $a^{n/2}$ and then use the formula in (b) to get $a^n \bmod m$. We then use the other formula in (b) and we get $a^{n+1} \bmod m$.Thus, by induction our algorithm works for all $n \in \mathbb{N}$.

- (Proof it Stops)

Let our potential function be b . WLOG let $b \neq 0, 1$ since we would halt immediately.Thus, if b is even then $b \mapsto b/2 < b$, and the minimum b can be without halting the program immediately is 2 thus $b - b/2 \geq 2 - 1 = 1$ If b is odd then $b - 1$ is even thus $b \mapsto (b - 1)/2 < b$. Again the min odd number without halting immediately is 3 thus $b - (b - 1)/2 \geq 3 - 1 = 2$

Our function is lower bounded by 0. Thus our functions stops.

- (Runtime Analysis)

We can ignore the base cases since they are all $O(1)$ operations. Now, if b is even then we call on our function on $b/2$, and this is the same in the odd case, and since we return after the recursive call we only make one each loop. The rest of the lines we can assume do $O(1)$ work thus we end up with the recurrence relation:

$$T(b) = T(b/2) + O(1)$$

Thus by the Master Theorem we get that $T(b) \in O(\log b)$

- (d) For $b = 0$ I would return the identity matrix, but overall I would not change anything the only thing I would have to implement is the matrix multiplication and the matrix modular.

The proof of correctness this is basically the same since we are given the equivalent statement in (b) but for matrices, and we can prove that:

$$\text{if } b \text{ is even, then } (A^{b/2} \mod m)^2 \mod m = A^b \mod m$$

the same way as we did in (b). Thus, since our proof of correctness relies on these two facts we can just do the same proof. Also, we have the same proof for halting. Finally, the runtime analysis is the same, but now we get the recurrence relation as:

$$T(b) = T(b/2) + O(n^3)$$

Since we would now do $O(n^3)$ work on lines 10,14,15. But since we assume that n^3 is a constant it becomes $O(1)$. Thus, we have the same relation thus $T(b) \in O(\log b)$

- (e) To find $F_k \mod m$. I would call my function in part (d) on the matrix:

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$$

where $b = k - 1$

(Proof of Correctness): I am going to use induction to prove that

$$A^k \mod m = \begin{pmatrix} F_{k-1} \mod m & F_k \mod m \\ F_k \mod m & F_{k+1} \mod m \end{pmatrix}$$

Base Case: ($k = 1$)

A^1 is correct just look at the matrix.

Inductive Step: Assume that this works for all $1 < k \leq n$ for some $n \in \mathbb{N}$. Consider $n + 1$.

Since by the formula we are given in (d) we know that

$$A^{n+1} \mod m = (A^n \mod m)(A \mod m) \mod m$$

By the inductive hypothesis we know that:

$$A^n \mod m = \begin{pmatrix} F_{n-1} \mod m & F_n \mod m \\ F_n \mod m & F_{n+1} \mod m \end{pmatrix}$$

$$A^{n+1} \mod m = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} F_{n-1} \mod m & F_n \mod m \\ F_n \mod m & F_{n+1} \mod m \end{pmatrix} \mod m$$

Thus for $A^{n+1} \mod m$ we get

$$\begin{aligned} A^{n+1} \mod m &= \\ &= \begin{pmatrix} F_n \mod m & F_{n+1} \mod m \\ (F_{n-1} \mod m + F_n \mod m) \mod m & (F_n \mod m + F_{n+1} \mod m) \mod m \end{pmatrix} \\ &= \begin{pmatrix} F_n \mod m & F_{n+1} \mod m \\ F_n \mod m & F_{n+2} \mod m \end{pmatrix} \end{aligned}$$

Thus, by induction we are done.

Now we can use the algorithm in (d) input the matrix A and input $b = k - 1$ and m . We can then use the proof of correctness of the algorithm to correctly get $A^{k-1} \mod m$, and then we just look at the bottom right entry which I proved will be $F_k \mod m$. Also, same proof for why it halts as in (d), and (b).

(Runtime analysis)

This is the same runtime analysis but now $b = k - 1$ thus we get the algorithm is $O(\log k)$ with the same proof that is in (b), and (d).

Problem 3

Algorithm 2 Max in circular shifted Array

- (a) **Require:** An array $A[1, \dots, n]$ of distinct integers that is sorted but has a circular shift.
Ensure: Finds the max element.
- ```

1: function FINDMAX(A)
2: if $|A| = 1$ then
3: return $A[1]$
4: $left \leftarrow 1$
5: $right \leftarrow n$
6: while $left < right$ do
7: $mid \leftarrow \lceil (left + right) / 2 \rceil$
8: if $A[mid] > A[(mid + 1) \bmod n]$ then $\triangleright$ Circular indexing for $mid + 1$
9: return $A[mid]$ $\triangleright$ Maximum found
10: if $A[mid] > A[left]$ then $\triangleright$ Left half is sorted, max is in the right half
11: $left \leftarrow mid + 1$
12: else
13: $right \leftarrow mid - 1$ $\triangleright$ Right half is sorted, max is in the left half
14: return $A[left]$ $\triangleright$ At convergence, $left = right$ and holds the max

```
- 

- (Proof of Correctness):

We just get rid of the case when  $A$  is a singleton set since it's one element is the max.

Consider the following cases:

Case 1: If  $A[mid] > A[(mid + 1) \bmod n]$ , then by the property of the circularly sorted array and that we have unique values, every element satisfies  $A[k] < A[(k + 1) \bmod n]$ , except for the maximum element. Therefore, if  $A[mid] > A[(mid + 1) \bmod n]$ , the maximum element is  $A[mid]$ , and the algorithm correctly returns it, and if this is not true then since we have distinct values this implies that  $A[mid] < A[(mid + 1) \bmod m]$  thus it is not the max. We now assume Case 1 did not happen.

Case 2: If  $A[mid] > A[mid]$ , the left half of the array is sorted. Specifically:

$$A[mid] < A[mid + 1] < A[mid + 2] < \dots < A[mid]$$

Since the left half is sorted, the transition point (where the maximum wraps to the minimum) cannot lie in the left half. Thus, we update the search to the right

half by setting  $mid \leftarrow mid + 1$ . We know assume Case 2 did not happen.

Case 3: If  $A[mid] \leq A[mid]$ , the right half of the array is sorted. Specifically:

$$A[mid] < A[mid + 1] < A[mid + 2] < \dots < A[right] < A[mid]$$

Since the right half is sorted, the transition point must lie in the left half. Thus, we update the search to the left half by setting  $right \leftarrow mid$ .

Finally, if the while loop terminates without returning (i.e., when  $left = right$ ), the range contains only one element, which must be the maximum. This is because each iteration eliminates half of the array, ensuring the maximum remains in the reduced range. When one element remains, it is guaranteed to be the maximum.

- (Halting):

Let our potential function be  $right - left$ . WLOG assume we don't get the case on line 6, and we end up with the one element case at the end, and that  $right \neq left$  since we would exit immediately.

If  $A[mid] > A[left]$  then our function becomes:

$$right - left \rightarrow right - mid - 1$$

Thus we get the difference is:

$$\begin{aligned} right - left - right + mid + 1 &= mid + 1 - left \\ &\geq (left + right)/2 + 1 - left \\ &= left/2 + right/2 + 1 \geq 1 + 1/2 \end{aligned}$$

since

$$left < right \implies left + 1 \leq right \implies right - left \geq 1 \implies right/2 - left/2 \geq 1/2$$

We can do this since we know  $right, left \in \mathbb{Z}_{>0}$

If  $A[mid] < A[left]$  then our functions becomes:

$$right - left \rightarrow mid - 1 - left$$

Thus we get the difference is:

$$\begin{aligned}
 right - left + left - mid + 1 &= right - mid + 1 \\
 &\geq right - (left + right)/2 + 1 - 1 \\
 &= right/2 - left/2 \geq 1/2
 \end{aligned}$$

Also, our function is lower bounded by 0 since it would stop if  $right = left$

Thus, our function halts.

- (Runtime Analysis): Lines 2, 3, and 15 involve constant-time operations and can be ignored for asymptotic analysis. The main contribution to runtime comes from the while loop.

Each iteration of the loop performs constant-time operations: computing  $mid$ , performing comparisons, and updating  $mid$  or  $right$ .

The search range is halved on each iteration because the algorithm either sets  $left = mid + 1$  and  $right = mid - 1$ . Thus, our new search is  $[left, mid - 1]$  or  $[mid + 1, right]$  and since  $right - mid \approx mid - left \approx (right - left)/2$  we half our search.

This gives the recurrence relation:

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

By the Master Theorem the runtime of the algorithm is  $O(\log n)$ .

---

**Algorithm 3** Find local Max

---

(b) **Require:** An array  $A[1, \dots, n]$  of integers.

**Ensure:** Finds the max element.

```

1: function FINDLOCALMAX(A)
2: if $|A| = 1$ then
3: return $A[1]$
4: if $A[1] > A[2]$ then
5: return $A[1]$
6: if $A[n] > A[n - 1]$ then
7: return $A[n]$
8: $left \leftarrow 2$
9: $right \leftarrow n - 1$
10: while $left < right$ do
11: $mid \leftarrow \lceil (left + right) / 2 \rceil$
12: if $A[mid] \geq A[mid + 1]$ and $A[mid] \geq A[mid - 1]$ then
13: return $A[mid]$ ▷ Local Maximum found
14: if $A[mid] < A[mid + 1]$ then ▷ To the right is increasing
15: $left \leftarrow mid + 1$
16: else
17: $right \leftarrow mid - 1$ ▷ To the left is increasing
18: return $A[left]$ ▷ At convergence, $left = right$ and holds a local max since a one

```

---

element set has a local max

• (Proof of Correctness):

Lines 2-6 are just getting rid of the end points and edge cases when we have a one element set which has a local max of that one element.

Consider the following cases:

Case 1: If  $A[mid] \geq A[mid + 1]$  and  $A[mid] \geq A[mid - 1]$ : This is by definition a local max, so we return and thus done. We now assume Case 1 did not happen.

Case 2: If  $A[mid] < A[mid + 1]$ : We then know that the array is increasing to the right and since it cannot increase forever since we have a finite set we know that it must have a local max in the interval or on the end. We know assume Case 2 did not happen.

Case 3: If  $A[mid] \geq A[mid + 1] \implies A[mid] < A[mid - 1]$  since we know Case 1 and 2 did not happen we know that  $A[mid] < A[mid + 1]$  or  $A[mid] < A[mid - 1]$  and since case 2 did not happen we know  $A[mid] < A[mid - 1]$ . Thus using the

same argument as in Case 2 we know there will be a local max in the left half of the array.

Finally, if the while loop terminates without returning (i.e., when  $left = right$ ), the range contains only one element, which must be the maximum. This is because each iteration eliminates half of the array, ensuring a local maximum remains in the reduced range. When one element remains, it is guaranteed to be a local maximum.

- (Halting):

Use the same function and same proof in (a) to show it halts.

- (Runtime Analysis):

This is the exact same runtime analysis as (a) with just 2 more checks before the function and the same amount of checks in the for loop, but since they are both constant time we get that the program is again  $O(\log n)$  for the same reasons as (a).

## Problem 4

- (a) Let  $C(i)$  be the number of ways you can reach the  $i$ th step of a staircase.

Base Cases:

For  $i = 0$  there is one way to get to the 0th step thus  $C(0) = 1$

For  $i < 0$  there are zero ways to get the  $i$ th step since we cannot go backwards thus  $C(i) = 0$  for  $i < 0$ .

Recurrence:

For the  $i$ th step for  $i > 1$  we get that:

$$C(i) = C(i - a) + C(i - b)$$

The reason is the only way to get to step  $i$  is either with an  $a$  step or a  $b$  step, and those can only come from the  $i - a$ th step and  $i - b$ th step.

- (b) Let  $M(i)$  be the most profit you can get from a rod of length  $i$ .

Base Cases:

For  $i = 0$  since we cannot sell any rod we get that  $M(i) = 0$

Recurrence:

To get the most profit of a rod of length  $i$  we get that:

$$M(i) = \max_{1 \leq j \leq i} (P[j] + M(i - j))$$

The reason this works is we look at all possible cuts of size  $j \leq i$  and consider the max of cutting at length  $j$  and selling it plus the max profit of selling a  $i - j$  long rod.

- (c) Let  $f(i, t)$  be true if we can get a sum of size  $t$ , with the first  $i$  elements and false otherwise.

Base Case:

For  $t = 0$  we can a sum of zero but taking the empty set.

For  $i = 0$  and  $t > 0$  this is always false, since we can't take any elements.

For  $t < 0$  we cannot get a sum of less than zero since we have non negative elements.

Recurrence:

$$f(i, t) = f(i - 1, t) \vee f(i - 1, t - S[i])$$

The reason this works is that to get the target sum  $t$  with the first  $i$  elements either we skipped the  $i$ th element, so we would just look at if we got  $t$  with the first  $i - 1$  elements, or we took the element thus we look at if we got  $t - S[i]$  with the first  $i - 1$  elements.

## Problem 4

(a) Base Cases:

For  $i = 0$  since we do not have to go anywhere for  $d_0 = 0$  we know that  $p(0) = 0$

Recurrence:

$$p(i) = \min_{0 \leq j \leq i} ((200 - d_i + d_j)^2 + p(j))$$

The reason this works is we look at all possibilities of going from point  $j$  to  $i$  and take the min of going to state  $j$  with the expression  $p(j)$  then we add the penalty of going from  $j \rightarrow i$ . We take the min of all possibilities to get the true minimum.

---

**Algorithm 4** Find min penalty from 0 to  $n$

---

(b) **Require:** An array  $A[1, \dots, n]$  of integers s.t.  $A[1] < A[2] < \dots < A[n]$

**Ensure:** Finds the min penalty from 0 to  $A[n]$

```

1: function FINDMINPENALTY(A)
2: if $|A| = 0$ then
3: return 0
4: $P[0, 1, \dots, n] \leftarrow [\infty, \infty, \infty, \dots, \infty]$ ▷ Array of length $n + 1$ with all ∞
5: $P[0] \leftarrow 0$ ▷ Base Case: $P[0] = 0$
6: for $1 \leq i \leq n$ do ▷ Iterate over all states
7: for $0 \leq j \leq i$ do ▷ Find best previous state
8: $P[i] = \min(P[i], ((200 - A[i] + A[j])^2 + P[j]))$
return $P[n]$ ▷ Return the minimum penalty to reach state n

```

---

- (Proof of Correctness):

We will prove the correctness of the FINDMINPENALTY algorithm by induction on  $i$ . The base case is  $P(0) = 0$ , which is correctly initialized as there is no penalty at the starting position.

For the inductive step, assume that for all  $k \leq i - 1$ , the computed values  $P[k]$  correctly represent the minimum penalty required to reach  $A[k]$ . The algorithm updates  $P[i]$  using the recurrence:

$$P(i) = \min_{0 \leq j \leq i} ((200 - A[i] + A[j])^2 + P[j]).$$

Since this equation correctly defines the minimum penalty needed to reach  $A[i]$ , and since the inner loop ensures that all possible transitions from previous states

$j$  are considered, the computed  $P[i]$  is optimal. The algorithm processes states in increasing order of  $i$ , ensuring that all dependencies are resolved before  $P[i]$  is computed. Thus, by induction, all values  $P[i]$  are correctly computed, and the final result  $P[n]$  is the minimum penalty to reach  $A[n]$ .

- (Runtime):

We can see from the two for loops that we will do:

$$O(1) + O(2) + O(3) + \cdots + O(n) = O(n^2)$$

To Find the path that we took to get to hotel  $n$  what we can do is take the array  $P$ , then we start at  $i = n$  and let  $j = i - 1$ . We then look for the  $j$  s.t.  $P(i) - P(j) = (200 - A[i] + A[j])^2$ . The first spot this is true we then set  $j = i$ , and we do the loop again till we get to zero. We would then mark the  $j$ 's for when this is true to find the path. The reason this works is we are just using the definition of the recurrence relation.

## Problem 5

Here, " $n(d) + 1$ " accounts for all integers  $1, 2, \dots, n(d)$  plus the possibility " $> n(d)$ ."

(a) With two blocks and  $d$  drops, one can handle up to

$$n(d) = \frac{d(d+1)}{2}$$

distinct heights. In other words,  $\frac{d(d+1)}{2}$  is the maximal  $n$  for which we can be sure of determining the LBH (or conclude  $> n$ ) within  $d$  total drops.

Strategy:

- Make the first drop from height  $d$ .
  - If the block breaks, then you have only 1 block left and  $d - 1$  drops remaining. You must do a linear search from height 1 up to height  $d - 1$  (which fits within  $d - 1$  drops).
  - If it does *not* break, proceed to a higher height.
- Make the second drop from height  $d + (d - 1)$ .
  - If it breaks, do a linear search in the interval  $(d, d + (d - 1))$ , using your remaining block and  $d - 2$  drops.
  - If it does not break, go higher.
- Continue in this fashion. Generally, after  $i$  drops (none of which have broken the block), you drop from height

$$d + (d - 1) + (d - 2) + \dots + [d - (i - 1)],$$

which is the  $i$ th partial sum of a decreasing sequence.

If after  $d$  such drops the block never breaks (i.e., the last tested height is  $\frac{d(d+1)}{2}$ ), we conclude  $k > \frac{d(d+1)}{2}$ . This strategy ensures all possibilities are covered.

We can see that the recurrence for  $n(d) = d + (d - 1) + \dots + 1 = \frac{d(d+1)}{2}$

(b) Why is this Optimal.

Intuitively:

- Once one block breaks, you are left with a single block. With only one block, you have no choice but linear search.

- If you have used  $r$  drops when the first break occurs, you only have  $(d - r)$  drops left. Linear searching  $(d - r)$  steps can only cover  $(d - r)$  more heights.
- Balancing the increments of your first block's drops as  $d, d - 1, d - 2, \dots$  ensures that in the worst-case scenario (where it breaks late or never breaks), you use all  $d$  drops but also maximize coverage.
- Assume that there is an algorithm  $\alpha$  that can test  $M$  more than  $d(d + 1)/2$ .

Let the algorithm drop first on floor  $x$ .

There are two outcomes:

It breaks then we have  $(d - 1)$  drops and 1 block left. To get the correct answer we are forced to do a linear search. Therefore, the only way to get the correct answer is to test from  $[1, x)$ . This implies that  $x - 1 \leq d - 1 \implies x \leq d$

It does not break. Then the max floor we can read is  $x + n(d - 1) \implies d \leq n(d - 1)$  because if our algorithm is to work every time we have to account for if it breaks at  $x$ .

We can then use an inductive argument to show that  $x + n(d - 1) \leq d + d - 1 + d - 2 + \dots + 1 \implies x + n(d - 1) = M \leq d(d + 1)/2$ . This is a contradiction of our assumption that  $M > d(d + 1)/2$ .

Hence,

$$n(d) = \frac{d(d + 1)}{2}$$

is indeed the maximum number of heights we can handle with 2 blocks and  $d$  drops.